
From Law to Action: Neuro-Symbolic Runtime Enforcement for MCP Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We describe a corpus-grounded neuro-symbolic implementation for constraining
2 Model Context Protocol (MCP) agents before protected tool execution. The method
3 starts from three different sources: multi-jurisdictional legal documents, CVE-
4 linked vulnerability and repair records, and SkillCenter Markdown procedures.
5 Source-preserving adapters produce Legal IR, Security IR, and Intent IR; these
6 share content identity and provenance but retain distinct meanings and authority.
7 Family-specific compilers expose first-order, frame, temporal-deontic, cognitive-
8 event, and solver-oriented views. An authorization service selects applicable
9 constraints, constructs per-action proof obligations, and returns a context-bound de-
10 cision. A separate enforcement adapter validates the live invocation and consumes
11 a permitted use before delegation. DuckDB and a typed Quack owner maintain
12 operational state; IPFS identifies and distributes immutable evidence; libp2p car-
13 ries peer traffic; and UCAN limits delegated capabilities. We document these
14 implemented roles, the source-to-IR transformations, the separation of learned
15 proposals from checked decisions, and the tests needed to establish guarded-path
16 safety. Corpus release statistics and inspected regression assertions are reported
17 separately from end-to-end evaluation, which remains to be completed.

18 1 Introduction

19 An autonomous agent can follow a useful procedure and still violate a legal restriction, reproduce
20 a known vulnerability, or act outside its delegated authority. A safeguard placed only in the model
21 prompt cannot distinguish these cases reliably. The implementation described here places source-
22 grounded constraints and a separate authorization boundary between proposed actions and protected
23 MCP handlers. MCP supplies tool discovery and invocation; application implementations remain
24 responsible for enforcing security requirements [1, 2].

25 Our method begins before a tool call. A legal collection supplies normative constraints, a vulnerability-
26 and-fix collection supplies security evidence, and a skill collection supplies descriptions of intended
27 procedures. These become three intermediate representations (IRs), not one undifferentiated retrieval
28 index. Formalization produces typed logic views and proof obligations. The runtime joins the results
29 against one exact invocation, then checks authorization again before the effect. This design combines
30 semantic compilation, proof search, capability security, and stateful execution rather than asking a
31 language model to certify its own behavior.

32 We document four implemented contributions: (1) source-preserving adapters and retrieval releases
33 for the three IR families; (2) multi-logic constraint compilation and proof-oriented authorization;
34 (3) DuckDB-backed supervision and exact-context enforcement; and (4) content-addressed, peer-
35 accessible evidence and delegated MCP execution. Legal interpretation and learned extraction remain

36 distinguishable from verified derivation. The paper is an implementation report and evaluation
37 template: source/card inspection does not establish a measured end-to-end safety or efficiency result.

38 **2 Corpus sources and their compilation into IRs**

39 **2.1 Legal documents to Legal IR**

40 The legal collection is an input to the reasoning system, not an application mentioned after the
41 architecture. Its source families include the United States Code, state statutes, state administrative
42 rules, Federal Register documents, state and local court rules, municipal and county codes, caselaw,
43 and national legislation outside the United States. The collection is expanding internationally;
44 complete worldwide coverage is a collection objective, not an assumption of the method. The
45 canonical registry binds each supported corpus to a country/branch, published dataset identifier, local
46 root, JSON-LD layout, Parquet layout, and source-specific identity fields.

47 The ingestion path preserves original documents separately from normalized provisions. Official
48 sources and archive-derived captures retain their origin, retrieval revision, extraction route, and
49 coverage limitations. For example, an international national-legislation snapshot can be recovered
50 from Common Crawl WARC and Wayback captures of an official publisher. That is a provenance-
51 bearing archive route, not a claim that an archive capture is a newly issued official text. Federal
52 Register proposals, notices, and final rules must likewise remain different source kinds. Legal
53 applicability depends on jurisdiction, authority, effective interval, definitions, exceptions, and the
54 facts of the request, not merely textual similarity.

55 The published retrieval layers add CID-keyed records, sparse indexes, vectors, and graph links. They
56 are views of a documentary substrate. A statutory citation edge is not a proved implication, an
57 embedding neighbor is not controlling authority, and a release named “IR” may contain retrieval
58 records rather than completed formal proofs. The corpus manifest must therefore count source
59 records, derived views, rejected records, and checked formalizations separately. Appendix A gives
60 source families, representative release pins, and the exact questions a coverage manifest must answer.

61 The legal compiler has two distinguishable paths. The rich deontic bridge preserves actor, action,
62 modality, conditions, exceptions, temporal expressions, definitions, references, available source spans,
63 and diagnostics. The stricter canonical compiler maps selected norms into a bounded vocabulary
64 with seven fields: modality, actor, action, object, conditions, exceptions, and temporal qualifiers.
65 Unrepresented material cannot disappear silently into a successful result. Source maps, declared
66 losses, producer/configuration identity, and review state travel with the derived object. Compilation
67 is thus reproducible and inspectable, while source-to-rule fidelity remains a separate validation
68 requirement. Executable-law work such as Catala supplies a relevant precedent, but our contribution
69 also includes runtime agent and capability boundaries [3].

70 **2.2 Vulnerability and repair evidence to Security IR**

71 The security source is not simply a list of CVE identifiers. CVEfixes links vulnerability records
72 to source repositories and fixes, supporting inspection of vulnerable and changed code [4]. Our
73 inspected release uses an exact pinned hitoshura25/cvefixes snapshot, preserves the original
74 Parquet bytes, and provides a source-CID-to-original-row index. The release reports 12,987 input
75 rows, 12,714 derived records, and 273 tombstones; these are release counts, not distinct-vulnerability
76 counts or verified-prohibition counts. Original evidence and derived outputs remain independently
77 addressable.

78 The conversion is explicit: a bounded source record and a policy candidate, together with their
79 review binding, enter `adapt_cvefixes_candidate`. The adapter constructs a canonical Security
80 IR declaration, including an exact scoped deny-policy candidate where representable. Its contract
81 rejects wildcard/generalized scope and attempts to insert an authority grant into the candidate.
82 Evaluation results remain separate from declarations. The formalization adapter then lowers the
83 declaration into policy, threat, transition, and claim views. The emitted prohibition binds action,
84 preconditions, effects, mitigations, language, scope, and CVE/CWE identifiers. A CVE or CWE
85 label alone is not the policy.

86 Vulnerable and fixed examples have different methodological roles. Vulnerable code is a positive
 87 control for a candidate security restriction; fixed code is a negative control for that particular restriction.
 88 A fix does not certify that all other vulnerabilities are absent. The formalization emits a candidate
 89 prohibition and proof obligations, not a theorem or an execution denial merely because a record was
 90 retrieved.

91 The runtime connection is concrete. `cve_security_gate` maps both Intent IR actions and generated-
 92 code fact groups into exact `SecurityAuthorizationRequest` objects under the same pinned Security IR
 93 root. It correlates the two streams. A harmless declared intent cannot mask an undeclared, broader,
 94 contradictory, or rejected effect in generated code. Incomplete mappings remain unknown. The
 95 hard-constraint compiler consumes these checks before quality or cost ranking; no favorable score
 96 offsets a failed security constraint.

97 2.3 SkillCenter Markdown to Intent IR

98 SkillCenter supplies procedural source material: `skill_md`, `library_md`, YAML metadata, titles,
 99 domains, profiles, original URLs, source identifiers, and licensing information. The bounded reader
 100 opens upstream SQLite bundles read-only and immutable, disables extension loading, checks expected
 101 tables and columns, and applies record-size limits. **SQLite here is an upstream corpus container,**
 102 **not the agent supervisor’s state database.** Markdown bodies are parsed as untrusted data; commands
 103 appearing in a skill are never executed by ingestion.

104 The pinned SkillCenter build reports 216,972 physical records in 24 bundles. The implementation
 105 explicitly reconciles physical counts against inconsistent upstream metadata. It computes two useful
 106 identities: an `entry_cid` over intrinsic record content and an exact body CID over UTF-8 `skill_md`.
 107 Container filename and dataset-revision provenance are retained separately so that repackaging
 108 unchanged intrinsic content does not change its entry identity. The derived release uses these
 109 identities across Parquet records, BM25 postings, graph nodes, vectors, and source lookup.

110 Intent IR describes what a procedure tries to accomplish. Its schema distinguishes goals, preconditions,
 111 postconditions, invariants, guards, effects, assumptions, failures, and verification steps. Actions refer
 112 to tools, inputs, outputs, and required evidence. Control edges distinguish next, success, failure,
 113 conditional, retry, parallel, and join relationships. Source spans and grounded-versus-inferred status
 114 accompany semantic nodes. A procedure’s word “permitted” is a source assertion, not a capability
 115 from the system owner.

116 The full retrieval corpus must not be confused with an exhaustively formalized skill library. The
 117 source adapter, schema, index builders, and formalizer/normalizer ports are inspectable; every claimed
 118 normalized document still needs its source and validation record. Learned normalization and a trained
 119 semantic encoder operate behind those ports. Corpus size alone says neither that every procedure has
 120 been normalized nor that every normalized procedure is safe to execute.

121 2.4 Common lineage without collapsing the domains

122 The three families share `ir_core` canonicalization, identity, provenance, schema, claim, obligation,
 123 and result contracts. They do not share an ontology or grant authority to one another. The kernel sep-
 124 arates declarations, runs, results, and receipts. Running a prover therefore creates a new result/receipt
 125 rather than mutating the underlying legal norm or skill declaration.

Table 1: Three source-to-IR paths. None of the source streams directly
 supplies execution authority.

Source stream	Derived semantics	Contribution to an action decision
Legal texts and versions	Norms, scope, authority, exceptions, temporal applicability	Which represented legal constraints apply?
CVE-linked code and fixes	Threat assumptions, effects, mitigations, scoped prohibition candidates	Does the proposed or generated behavior violate a reviewed security constraint?

Source stream	Derived semantics	Contribution to an action decision
Skill Markdown and metadata	Goals, actions, guards, effects, verification, control flow	What is the agent trying to do, and what effects does its procedure require?

126 Sparse, dense, and graph retrieval nominate evidence; hard applicability and exact identity determine
127 whether it may be used. The SkillCenter release demonstrates bounded fetching through a manifest,
128 lexical or centroid meta-index, selected Parquet shards, and final CID-keyed records. This reduces
129 repeated full-corpus transfer. It is an engineering mechanism for efficient context construction, not a
130 substitute for dependency coverage or a measured end-to-end token-saving result.

131 3 Logic families and executable proof work

132 3.1 Why the implementation has several logics

133 An IR is a domain representation; a logic family determines how its statements are interpreted. A
134 prover is a backend that checks a supported encoding. Keeping these concepts separate is essential. A
135 temporal obligation cannot become an ordinary Boolean predicate without recording what temporal
136 meaning was lost. The compiler therefore produces native views, shared symbol bindings, source
137 maps, and explicit cross-view links rather than concatenating formulas from different logics.

Table 2: Roles of native views and checking infrastructure. A family’s presence in a registry does not mean every formula or backend is qualified.

Logic or formal view	Implemented role in the stack	Agent-safety use
First-order logic and F-logic/frame views	Predicates, quantified relations, typed objects, classes and slots	Describe actors, resources, tool interfaces, and relations needed by a rule
Deontic logic	Obligation, permission, prohibition; richer domain slots where supported	Separate required, allowed, and forbidden behavior
Temporal logic and TDFOL	Quantification plus deontic and temporal operators	Validity windows, prerequisite order, continuing duties and deadlines
Event calculus and DCEC	Events/fluents and cognitive/normative structure	Relate actions to state changes; distinguish an agent’s belief or intention from established facts
Rule/policy and Datalog-style views	Bounded derivation and authorization declarations	Compute applicable facts and grants under an explicit world policy
Hoare-style contracts and SMT	Preconditions, postconditions, transitions, arithmetic and typed constraints	Check generated code and effect invariants in a declared fragment
Interactive proof kernels	Reconstructed Lean/Coq/Isabelle proofs through configured adapters	Independently check proof candidates when the required assurance demands it

138 Temporal Deontic First-Order Logic (TDFOL) explicitly combines first-order predicates and quanti-
139 fiers with obligation/permission/prohibition and temporal operators such as always, eventually, next,
140 until, and since. Deontic Cognitive Event Calculus (DCEC) adds cognitive operators including belief,
141 knowledge, intention, desire, and goal alongside event and normative structure. These are useful
142 distinctions for agents: a model’s belief that a request is authorized is not an authenticated fact that it
143 is authorized. Encoding a knowledge operator does not validate a perception result or make generated
144 text true.

145 F-logic contributes object-centered semantic relations, not digital signatures. Event calculus is the
146 reasoning vocabulary for events and changing fluents; the append-only execution Event DAG is its

147 provenance substrate, not itself an event-calculus proof. Higher-order, linear, and separation tags also
148 occur in the broader training/representation contracts. We do not infer live proof support from those
149 tags. Appendix B records the role, supported scope, and appropriate evidence boundary for each
150 family.

151 3.2 Translation, solving, reconstruction, and reuse

152 Each backend request binds its family, premises, assumptions, target obligation, supported fragment,
153 and resource limits. Z3 and cvc5 supply SMT results, models, or cores; Vampire and E serve first-
154 order automated proving paths; interactive theorem-prover adapters support proof reconstruction and
155 kernel checking. Hammers select premises, translate to TPTP or SMT-LIB, run candidate-producing
156 portfolios, and reconstruct a native proof where supported. Solver success and kernel acceptance
157 remain different evidence kinds.

158 The verification API also exposes abstract interpretation, assume-guarantee discharge, incremental
159 SMT, differential checking, interpolation, and counterexample handling. These are selected methods,
160 not a checklist run on every invocation. For example, the inspected interpolation path is bounded to
161 QF_LIA with cvc5 production and independent checks of both interpolation conditions and shared
162 vocabulary. An unavailable interpolant can produce a separately labeled validated core; it cannot be
163 reported as a successful interpolant.

164 Proof-grounded IR learning uses source/IR, translation, reconstruction, proof, tactic, and coun-
165 terexample traces. Its contracts distinguish structural equality, logical equivalence, entailment,
166 equisatisfiability, and heuristic similarity. A learned encoder can nominate a representation or
167 premise set; the declared checker supplies acceptance evidence. Translation and reconstruction
168 receipts retain the weakest relevant boundary. A nearest-neighbor score, a successful paraphrase, or a
169 corpus-membership proof cannot upgrade a candidate to a theorem.

170 4 Joining intent, law, and security before execution

171 4.1 Concrete invocation and hard-constraint admission

172 An `InvocationIntentEnvelope` commits to source identity, tenant, actor, audience, delegation,
173 tool/version, arguments, proposed effects, environment, nonce, and relevant roots. Trusted adapters
174 bind the effect description to the actual operation. Otherwise an agent could authorize a harmless
175 description and execute a different effect.

176 Legal and security applicability are evaluated independently. Legal queries bind jurisdiction, subject,
177 authority, time, actor, resource, definitions, cross-references, exceptions, and source integrity. Security
178 queries bind principal, capability, trust zone, asset, channel, action, effect, failure/rollback, and
179 environment evidence. Hard filters precede ranking. Missing or contradictory required evidence is
180 retained as a blocking condition rather than removed to make a request pass.

181 The supervisor's `IRConstraintCompiler` joins these results against the exact candidate ac-
182 tion/effect graph. It rejects undeclared effects, unmatched domain bindings, incomplete legal applica-
183 bility, applicable prohibitions, unresolved obligations, denied/unknown/conflicting security decisions,
184 stale roots, unsatisfied dependencies, missing proofs, and failed validation. Intent conformance and
185 legal permission constrain the candidate but do not grant security authorization. The output includes
186 rejection witnesses and the affected action set for local replanning rather than a scalar score that a
187 model can optimize around.

188 4.2 Authorization proof jobs and decision receipts

189 `AuthorizationQueryComposer` generates per-action jobs for applicability, positive grant, non-
190 conflict, security invariant, pre-obligation, coverage, and context binding. Coverage is relative
191 to the declared modeled scope, not proof that all relevant law has been collected. Additional
192 jobs can address consistency, translation, reconstruction, and obligations during or after execu-
193 tion. `AuthorizationPortfolio` records attempts and selects compatible results deterministically;
194 contradictory authoritative results require review.

195 IntentAuthorizationService composes normalization, lowering, evidence selection, jobs, port-
196 folio execution, decision, receipt, and optional internal capability derivation. It does not execute the re-
197 quested tool. Unknown, unsupported, contradictory, SAT-only, simulated, and merely policy/evidence-
198 ready outcomes cannot silently satisfy the proof-oriented allow policy. An AttestedProofVerifier
199 checks selected evidence against exact statements, assumptions, source/build/toolchain identities,
200 obligations, scope, and corpus/policy/revocation roots. Signed assertions and zero-knowledge artifacts
201 retain the statement and verifier they actually attest.

202 4.3 Pre-invocation enforcement

203 A receipt becomes operational only through a protected effect boundary.
204 SupervisorPreInvocationEnforcement.authorize_and_delegate in enforce mode
205 rechecks the live invocation, outcome, effects, expiry, roots, and capability, then atomically consumes
206 the permitted use before invoking the delegate. Off, audit, and shadow modes do not provide the
207 same prevention semantics. The supplied clock, roots, receipt issuer, and consumption store must be
208 trusted and appropriate to the deployment.

209 The separate short-lived ExecutionPermit path also binds the admitted graph, roots, mandatory
210 dependency closure, validation plan, caller, policy, lease, fence, expiry, and use bound. Plan admission
211 is not execution permission, and permission is not completion evidence. These interfaces must be
212 traced at each entry point: a lightweight Profile D dispatcher, a UCAN-gated MCP server, and a
213 full proof-oriented wrapper have related but nonidentical checks. Our security claim concerns a
214 configured, mediated path, not every historical compatibility route.

215 5 DuckDB and event-driven operational state

216 The supervisor’s mutable control state is **DuckDB**, specifically `control.duckdb`, behind the typed
217 Quack state-owner path. This is distinct from immutable IPFS artifacts and from read-only source-
218 corpus containers. DuckDB/Quack provides the database access substrate [5]; the implementation
219 adds ownership, command validation, generation/fence checks, and bounded retries.

220 QuackStateClient exposes closed, named parameter-bound commands rather than model-supplied
221 SQL. The inspected client supports an embedded exclusive-lock mode for hermetic/single-process
222 tooling and a Quack-mode typed Unix-socket owner gateway. It verifies store/server identity, schema
223 fingerprint, generation, and extension fingerprint. Generic client ATTACH and arbitrary SQL are not
224 the model-facing interface. The operational boundary is the project’s typed service, not unrestricted
225 access to every feature supported by the underlying Quack extension.

226 StateTransaction binds expected store generation, fence epoch, row revision, and idempotency
227 identity in short transactions. Same-row optimistic conflicts can receive bounded jittered retries;
228 stale generation, mismatched fence, and changed-payload idempotency conflicts are non-retryable
229 authority failures. A lost reply resolves against the committed command identity rather than causing
230 the LLM to repeat an effect. Database transitions are authoritative after commit. Event/outbox/cursor
231 integration and recovery must be qualified for the selected owner deployment; they are not replaced
232 by Markdown task-board edits.

233 Provider processes do not receive the owner’s token or an open SQL capability. The threat model
234 prohibits extension installation, file-reader escapes, cross-root operations, and Python UDF surfaces
235 through the typed client. Offline maintenance requires exclusive ownership. This is a single-owner
236 failure domain, not multi-primary consensus. DuckLake projections or exported reports can support
237 analysis but do not acquire task-mutation authority.

238 Source, proof, policy, capability, and completion events trigger rescanning and targeted invalidation.
239 Relevant changes reopen affected obligations; unchanged semantic dependencies remain candidates
240 for reuse. The supervisor’s generation and fencing mechanisms prevent an old worker from publishing
241 its result over newer state. Durable capability consumption and uncertain remote-effect recovery
242 must be tested in the actual executor: a DuckDB commit alone does not make an arbitrary remote
243 API exactly once.

244 6 IPFS, libp2p, UCAN, and MCP++ in the execution path

245 6.1 IPFS: identity and distribution of evidence

246 The InterPlanetary File System (IPFS) uses content addressing to name bytes by content rather than
247 by a mutable location [6]. Kit stores source bodies, IR declarations, graphs, ContextPacks, proofs,
248 source/alias maps, decisions, and receipts as immutable artifacts. A recipient rehashes the retrieved
249 bytes under the declared profile. Canonical declarations bind domain and schema so equal-looking
250 payloads from different families cannot substitute for one another.

251 The IR kernel uses a declared canonical-JSON identity envelope and CIDv1/raw/SHA-256 profile;
252 other contracts, including Profile D artifacts, use DAG-JSON. The codec and preimage are part
253 of the contract. A plain SHA-256 digest is not relabeled a CID. A graph root commits to its
254 referenced closure, while mutable current roots advance through a controlled publication path. This
255 enables reproducible evidence reuse across machines. Availability still requires retention/pinning and
256 functioning peers; local CID computation proves neither publication nor public availability. Private
257 evidence needs explicit access and confidentiality controls.

258 6.2 libp2p: peer communication, not permission

259 libp2p supplies peer identities, transport selection, secure channels, protocol negotiation, and multi-
260 plexed streams [7]. A Peer ID identifies a key-bound peer; it is not an application user or a tool grant.
261 The MCP++ P2P carriage profile uses /mcp+p2p/1.0.0 with framed MCP messages. Stdio, HTTP,
262 and P2P adapters should converge on the same validated dispatch semantics rather than implement
263 different authorization policies.

264 The relevant architectural benefit is that another agent can exchange an objective, request an IR
265 artifact by CID, obtain proof evidence, or call a permitted tool without sharing the supervisor database.
266 Discovery or a successfully authenticated peer connection does not authorize those operations. Tests
267 of a local stream framing path also must not be reported as a wide-area network or adversarial
268 transport qualification.

269 6.3 UCAN: attenuated delegated authority

270 The User Controlled Authorization Network (UCAN) represents signed delegated capabilities [8]. In
271 the inspected strict verifier, Ed25519 signatures, issuer/audience continuity, parent proofs, explicit
272 resource/ability fields, validity windows, and supported request bounds are checked. A child capability
273 must remain within the parent's authority. A complete path segment matters: `tenant-a/*` must not
274 accidentally include `tenant-ab`. The durable revocation/replay ledger supplies state that a signature
275 alone cannot provide.

276 This is separate from a formal proof. UCAN answers whether an authenticated principal may exercise
277 a delegated capability. Legal and security reasoning answer whether this particular operation satisfies
278 the applicable modeled constraints. Both must be satisfied where required. A parsed unsigned
279 delegation stub, an agent-authored permission sentence, or a valid signature on irrelevant bytes cannot
280 replace the strict verifier. The proof-derived AuthorizationCapability and a signed UCAN token are
281 also distinct formats; deployments must connect them through an explicit trusted binding rather than
282 equating their names.

283 6.4 MCP++: where these mechanisms meet

284 MCP++ adds optional profiles around MCP rather than replacing JSON-RPC. Profile A identifies the
285 interface; B binds invocation and result artifacts; C specifies delegation; D carries temporal-deontic
286 policy; E carries P2P messages; F records causal provenance; and G describes tasks, claims, leases,
287 and coordination. Their portable schemas support heterogeneous agents, while the implementations
288 perform the checks.

289 Kit's protected `tools/call` route invokes AuthorizationGate before dispatch. The gate checks
290 envelope/tool/resource/ability bindings, a concrete UCAN verifier and ledger, and the Datasets-
291 backed Profile D provider, then records the decision. Missing validators or required authorities
292 deny. Profile D matches actor/action/resource/time, applies prohibition precedence, and requires a

293 matching permission. A returned obligation needs a handling policy; the inspected strict Kit gate
294 accepts plain allow, not unresolved allow_with_obligations. This is a concrete policy gate.
295 Stronger theorem-required actions additionally need the proof-oriented enforcement path; carrying a
296 proof_cid in an output envelope does not establish that it was checked before execution.

297 7 Worked source-to-effect trace and evaluation

298 Consider an agent that retrieves a Markdown procedure for exporting records. The following is a
299 composition walkthrough, not a newly executed integration result. The source record contributes
300 a procedure and source spans, not permission. Intent IR describes the selected records, recipi-
301 ent/channel, expected effects, prerequisites, and verification. The legal slice supplies applicable
302 reviewed disclosure/retention constraints. A retrieved vulnerability/fix example contributes a scoped
303 candidate constraint for an unsafe data-flow or missing validation; applicability and review determine
304 whether it participates. Generated code is independently analyzed so that its effects can be correlated
305 with the stated intent.

306 The supervisor binds this candidate to current Legal/Security/Intent roots and composes the required
307 proof jobs. An unapproved destination, an unresolved exception, or an undeclared code effect blocks
308 admission. A permissible candidate still requires a current grant for the exact actor, tool, resource,
309 and bounds. The protected runtime checks the receipt and capability immediately before delegation;
310 a recipient change invalidates the argument commitment, while revocation or consumed-use state
311 rejects replay. DuckDB records operational progression, and immutable evidence records explain
312 which sources, rules, proofs, and grants led to the result. The model can propose a revised plan, but it
313 cannot remove a failed obligation by rewriting its explanation.

314 The evaluation should exercise these existing paths. Source tests cover bounded SkillCenter reading,
315 deterministic identities, CVE scope and authority rejection, typed compilation, proof-result authority,
316 enforce-mode non-delegation, replay/concurrent consumption, Quack command/fence checks, and
317 MCP transport parity. Inspected test assertions are not executed results. The next frozen experiment
318 must record actual results separately for fixture, local solver, cryptographic, storage, and end-to-end
319 agent paths.

320 Compare unguarded sandbox execution, retrieval plus prompt instructions, lightweight policy/UCAN
321 gating, and full proof-oriented enforcement on the same handlers. Cross-source cases should
322 include malicious skill text, a wrong-jurisdiction or stale legal version, a misleading CVE match,
323 vulnerable/fixed controls, omitted code effects, a widened capability, changed arguments, and restarted
324 state. Measure actual forbidden delegate effects together with allowed-task success, false denial,
325 abstention, source/formalization fidelity, route coverage, tokens, proof cost, and latency. No numerical
326 safety or cost advantage is asserted before these runs. Appendices provide the lineage and experiment
327 fields.

328 8 Scope and conclusion

329 The implemented contribution is a source-grounded constraint pipeline and a controllable action
330 boundary. It does not require a complete global corpus to enforce a reviewed bounded policy, and a
331 large corpus does not prove completeness. Formalization may be partial; model proposals, signatures,
332 content identities, monitor observations, solver results, and kernel proofs retain distinct meanings.
333 Complete mediation, trusted current roots, authenticated issuers, and appropriate persistence remain
334 deployment requirements.

335 The system is best understood as a compiler-and-runtime stack: Legal IR supplies normative con-
336 straints, Security IR supplies behavior and threat constraints, and Intent IR supplies proposed
337 procedures. Native logics and proof tools determine what evidence is needed; DuckDB maintains
338 operational truth; IPFS preserves exact artifacts; libp2p carries peer interactions; and UCAN limits
339 delegated authority. The safety-relevant result is not a more confident explanation. It is whether a
340 non-admitted action is prevented from reaching a protected handler.

References

- [1] Model Context Protocol contributors. Model Context Protocol specification, revision 2025-11-25, 2025. URL <https://modelcontextprotocol.io/specification/2025-11-25>. Accessed 6 September 2026.
- [2] Model Context Protocol contributors. Model Context Protocol: Authorization, revision 2025-11-25, 2025. URL <https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization>. Accessed 6 September 2026.
- [3] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP):77:1–77:29, 2021. doi: 10.1145/3473582.
- [4] Guru Bhandari, Amara Naseer, and Leon Moonen. CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2021. doi: 10.1145/3475960.3475985.
- [5] DuckDB team. Quack: The duckdb client-server protocol, 2026. URL <https://duckdb.org/2026/05/12/quack-remote-protocol>. 12 May 2026. Accessed 6 September 2026.
- [6] IPFS contributors. Content identifiers (CIDs), 2026. URL <https://docs.ipfs.tech/concepts/content-addressing/>. Documentation accessed 6 September 2026.
- [7] libp2p contributors. libp2p connections specification, 2026. URL <https://github.com/libp2p/specs/blob/master/connections/README.md>. Specification accessed 6 September 2026.
- [8] UCAN contributors. User controlled authorization network (UCAN) specification, 2026. URL <https://ucan.xyz/specification/>. Specification accessed 6 September 2026.

364 A Corpus lineage and release accounting

365 A.1 Legal source families and acquisition routes

366 The author-maintained legal program includes public releases for the U.S. Code, state statutes, state
 367 administrative rules, Federal Register documents, state/local court rules, municipal/county codes,
 368 caselaw, and national legislation. It also includes international harvests and retrieval releases. The
 369 paper does not interpret a dataset title, a country registry entry, or a successful scrape as exhaustive
 370 coverage. The private source map records identifying dataset names and inspected revisions; a
 371 submission artifact should substitute anonymous manifest identifiers while preserving the actual
 372 records and lineage.

Table A1: Legal source families and provenance requirements. Identifying publisher/dataset mappings belong in the private author record until camera-ready.

Source family	Representative inspected substrate	Required distinctions
Federal statutory law	U.S. Code sparse GraphRAG release, pinned source revision and official release point	Codification/release point, provision identity, amendment history and actual effective dates
State law	State-statute release and separate administrative-rule corpus	State, authority family, rule/statute distinction, missing jurisdictions and time coverage
Federal regulatory material	Federal Register collection	Proposed rule, final rule, notice, publication date, effective date and codified regulation are not interchangeable
Court rules and caselaw	Merged per-state court rules; Caselaw Access Project-derived collection	Court, jurisdiction, local/general rule, opinion treatment, holdings versus quotations
Municipal and county law	Multi-drop municipal harvest	Publisher, jurisdiction, per-drop discovery and failure counts; registry aliases may lag new release namespaces
International legislation	Netherlands corpus; Belgian official-source archive harvest; Argentine CID-keyed retrieval release	Language, source authority, consolidation date, archive capture, extraction version and incomplete coverage

373 The inspected U.S. Code release card pins a source revision and the release point `us/p1/118/45`;
 374 it must not be described as every later law in force. The state-law card similarly provides a pinned
 375 source revision, a release label, and a vector model revision. These are reproducibility fields, not
 376 automatic temporal-validity guarantees. The registry’s local fields also vary by corpus: a field named
 377 `id` or `jurisdiction` in one source must not be silently interpreted as the same CID or state-code
 378 field used in another.

379 The municipal card inspected on 6 September 2026 reports 567 jurisdictions across four drops. Its
 380 fourth drop reports 300 new jurisdictions and 448,090 HTML sections across 41 states. Those
 381 section counts describe that drop, not every municipality or necessarily the entire merged corpus. The
 382 Netherlands release reports 4,999 laws and 89,737 articles, with additional index/embedding rows.
 383 The Belgian card reports incomplete coverage of 9,116 instruments plus 282 leftovers and identifies
 384 an archive-based acquisition route. These card-reported quantities have not been independently
 385 recounted here and must be frozen or remeasured for the submitted experiment.

386 A legal source-to-rule manifest should retain original bytes, source URI, retrieval method, acquisition
 387 time, instrument and provision identifiers, source span, language, jurisdiction, authority, effective
 388 interval when known, parser version, normalized-text identity, parent/source revisions, rejected rows,
 389 licenses, and review status. Cross-references, definitions, exceptions, and competing interpretations

require explicit records. A translation from source into a restricted seven-field grammar must retain unsupported facets outside that grammar and block uses that require them.

A.2 Security source release and control construction

The inspected derived security release pins hitoshura25/cvefixes at d4f5c4ea65329d9ccbb8a3b3149e5d06eda5edb2. This is the exact input snapshot; it must not be replaced in the narrative by the statistics of a different upstream CVEfixes release. The release reports 12,987 original rows, 12,714 derived records, 273 tombstones, 85,169 graph nodes, and 167,364 graph edges. Graph nodes, edges, vectors, source rows, and unique CVEs are different populations.

The release retains byte-identical original Parquet, per-file digests/CIDs, row counts, and an original-row locator. The locator maps a source identity to a copied source shard and local row position. BM25, vectors, graph adjacency, and a retrieval manifest make the evidence remotely addressable without downloading all source bodies. Their identities permit integrity checks; none establishes that a vulnerability generalizes to another program.

The domain adapter receives a typed PolicyCandidate, its SourceRecord population, and review binding. It rejects generalized/wildcard scope, unknown authority assertions, duplicate JSON keys, and noncanonical fields on the relevant paths. It preserves a declaration and the detached candidate/review evidence separately. The formalizer specializes the shared Security IR policy view using exact typed terms for action, preconditions, effects, mitigations, language, scope, and CVE/CWE classifications. It emits a deontic prohibition candidate with explicit non-authoritative flags.

Evaluation must include both the original vulnerable condition and the fixed negative control. A mitigation that removes one unsafe data-flow does not imply universal safety. Transfer to a proposed tool or generated patch needs an explicit mapping, a supported behavior model, applicable scope, and a checked rule. The runtime cve_security_gate separately maps intent-side and code-side facts, rejects undeclared or broadened effects, and preserves unknown mappings. The selected gate result must be bound to the same action and Security IR root used by plan admission.

A.3 SkillCenter source identity and procedure representation

The inspected SkillCenter source is Tommysha/skillcenter-bundles at revision f9dd4fec3c86d85ebf116c7408ac5ce602c418a1. The repository build notes report 216,972 physical skill records across 24 bundles, rather than relying on the upstream card total. Expected source tables are bundle_meta, skills_index, and skills_content; source bodies include skill_md and library_md. SQLite is read as an untrusted, immutable input container. It is not the authoritative supervisor database.

SkillCenterSkillRecord retains domain, profile, title, language, source URL/type/IDs, metadata YAML, Markdown, dataset revision, bundle filename, and bundle digest. Its intrinsic identity excludes the packaging provenance while retaining the source fields that define the entry. An exact body CID separately identifies the Markdown bytes. Thus a record can be traced to a bundle yet retain its intrinsic identity if the same content is repackaged. License expression and risk remain available for source-use policy.

The schema of IntentIRDocument distinguishes statements from actions and control edges. Statement roles include goal, precondition, postcondition, invariant, guard, effect, assumption, failure, and verification. Action references include tools, inputs, outputs, objects, preconditions, effects, and verification steps. Edges include next, on-success, on-failure, conditional, retry, parallel, and join. Each node can be source-grounded or inferred and carries review state. These fields permit the system to ask what an agent proposes, what must already hold, and what evidence should follow.

The published retrieval release reports 216,972 vectors corresponding to canonical entries; it also contains millions of lexical/index rows. Index totals must not be called additional skills. The bounded Parquet release, centroid routing, and paged graph navigation are retrieval mechanisms. Counts of available records do not establish counts of normalized Intent IR documents, valid procedures, safe skills, or learned models trained successfully.

439 A.4 Record chain and minimum provenance

pinned source revision + raw artifact bytes
 -> bounded source record + content identity + use policy
 -> LegalIR | SecurityIR | IntentIR declaration
 -> native formal views + source map + loss/coverage diagnostics
 -> selected applicable constraints + exact invocation
 -> proof jobs + checked evidence + decision receipt
 -> authenticated capability / execution permit
 -> guarded effect + operational observation + durable record

440 Each arrow needs a producer/configuration identity and input/output references. Declaration, valida-
 441 tion, admission, and effect observation are separate steps. Source bodies, local files, indexes, and
 442 public releases need their own source-access and retention rules; a content address does not provide
 443 encryption or license rights.

444 B Logic-family and backend method map

445 B.1 Native semantics

Table B1: Logic families have different meanings. A shared symbol table or cross-view link does not prove that two encodings preserve those meanings.

Family/view	Representation and operations	What the paper may claim
Propositional and first-order	Boolean operators, typed terms, predicates, variables and quantifiers	Bounded facts and implications under the encoded vocabulary; not general natural-language understanding
F-logic / frame	Object identity, class membership, slots and relational inheritance	Structured semantic relations for source entities, agents, tools and resources; not a cryptographic or temporal proof by itself
Deontic	Obligation, permission, prohibition; richer DCEC domain operators	Explicit normative force under the selected axioms/profile; an encoded permission is not an issued capability
Temporal / TDFOL	Always, eventually, next, until, since; quantified normative formulas	Model time-sensitive requirements and ordering; a backend must support the actual formula fragment
Event calculus	Events, changing fluents, initiation/termination and persistence as represented	Reason about transitions and obligations over traces; an observed event record is not a proof of every transition
DCEC	Cognitive operators for beliefs, knowledge, intentions, desires and goals, plus normative/event structure	Keep agent attitudes separate from asserted world facts; fact grounding still needs evidence
Datalog / rule-policy / SecPAL-like rendering	Bounded rules and authorization declarations where selected	Rule derivation and policy checking with an explicit open/closed-world convention, not automatic theorem authority
Hoare-style / transition / SMT	Preconditions, postconditions, abstract state and theory-specific constraints	Check program/guard properties under stated assumptions and available arithmetic, array, string or other fragments
Higher-order and interactive proof	Native proof terms/scripts and dependent/higher-order constructs supported by the chosen prover	Kernel-checked statements in that prover and environment, not universal translation correctness

Family/view	Representation and operations	What the paper may claim
Linear / separation	Declared broader training/representation families	Mention as registered or staged where appropriate; do not claim a live checker without a concrete supported translation and execution receipt

446 The TDFOL and DCEC Python cores establish parsable native representations. Compiler bridges and
447 family engines determine which source constructs they can lower. Resource-scoped authorization
448 may be representable without invoking every family. The manuscript should state which view was
449 selected per experiment rather than treating the complete registry as one solver.

450 B.2 Proof workflow and result classes

451 The common proof workflow is: choose a supported view; create a typed obligation and premises;
452 select a backend/hammer plan; execute within a bound; preserve translations, models, cores, and
453 proof candidates; reconstruct where required; run the declared checker; then emit a result and receipt.
454 Z3/cvc5, Vampire/E, and Lean/Coq/Isabelle belong to different stages. A timeout or unavailable
455 backend is not a refutation, and satisfiability of a favorable instance does not prove universal safety.

456 Datasets distinguishes authority kinds such as theorem proof, satisfiability, runtime monitor, evidence
457 readiness, and policy approval. These are non-interchangeable. The supervisor also uses an assurance
458 classification for whether the evidence meets a requested threshold. Neither mechanism permits a
459 provider to upgrade its own output by naming it “verified.” A cryptographic attestation must bind the
460 actual checked claim, verifier and public inputs; proving corpus membership alone cannot prove the
461 member’s theorem.

462 Counterexample-guided refinement identifies missing facts or assumptions in the *encoded* problem
463 and requests specific additional context. Interpolation and unsat cores can narrow that request; they
464 do not discover an omitted statute automatically. Abstract interpretation overapproximates a qualified
465 program fragment, while assume-guarantee discharge checks that a producer’s established guarantee
466 entails a consumer’s assumption. Learned premise/tactic selection and proof-grounded encoders
467 improve search proposals; independent checking remains the acceptance operation.

468 C Database, artifact, network, and capability boundaries

469 C.1 Storage is not one undifferentiated database

Table C1: Database and protocol responsibilities. Legacy or auxiliary indexes can exist without changing the canonical DuckDB control-plane description.

Layer	Technology and inspected role	Incorrect inference to avoid
Supervisor operational authority	DuckDB control.duckdb; typed Quack owner/client; StateTransaction CAS and idempotency	“The supervisor uses SQLite” or “each agent writes the database file directly”
Source input packaging	Read-only SkillCenter SQLite bundles; source Parquet/JSON-LD/HTML/WARC	“An input database format defines the runtime architecture”
Retrieval products	BM25, vectors, graphs and bounded Parquet shards keyed by source IDs/CIDs	“A retrieved neighbor is a grant or a proof”
Immutable artifacts	Kit-managed content-addressed blocks, manifests, maps, proof/decision receipts, optional IPFS replication	“The newest block is automatically the accepted state”
Peer carriage	libp2p negotiated streams and MCP++ framing	“A peer identity grants tool permission”

Layer	Technology and inspected role	Incorrect inference to avoid
Delegated capability	Strict signed UCAN verification and revocation/replay state	“A valid token proves the action is legally and semantically safe”

C.2 A state transition through the typed owner

A caller supplies a named command and typed parameters, not a SQL string. The owner validates store identity, schema/generation, task revision, fence, and idempotency key. A successful transaction commits the permitted state change and its recorded command result. The identical command can retrieve that result after a lost reply; a changed payload with the same identity is a conflict. Stale generations and fences require new admission rather than blind retry. Event delivery can wake dependent work, but does not substitute for the committed state.

The inspected QuackStateClient describes both an embedded mode and a typed owner-gateway mode. Documentation that discusses generic Quack/ATTACH capability must not override the current client’s deliberately narrower interface. Likewise, the architecture should not promise that a reference in-memory capability consumption store is already backed by DuckDB. A deployment must supply and test the actual durable adapter used for receipt/capability consumption.

C.3 IPFS and libp2p are complementary

IPFS answers which artifact bytes are requested and enables verifiable retrieval of that content. Canonical IR, a source document, a decision receipt, and a graph node can each have independent identities. Pin/retention and transport choices determine availability. Public availability is not the default requirement for private legal or security material.

libp2p answers how peers connect, authenticate a peer key, negotiate an application protocol, and exchange framed streams. A protocol ID selects the decoder; it does not select a legal jurisdiction or an authorization profile by itself. TLS/Noise or native transport security protects a channel when configured; it does not turn a peer’s proof claim into a checked proof. Deployments must record actual transports and security settings. Do not count an in-process call to a P2P handler as a full libp2p network experiment.

C.4 UCAN and proof-oriented capabilities are distinct

Strict UCAN checks cryptographic issuer identity, delegation continuity, explicit resource/ability bounds, temporal validity, permitted attenuation, revocation, and replay policy. It does not infer delegation from natural-language text. A principal may possess a valid capability yet fail a legal or security obligation. Conversely, a mathematically valid proof of a property does not authorize the principal to invoke an administrative tool.

The proof-oriented AuthorizationCapability is an additional exact-context contract. Its relationship to a UCAN envelope must bind the same actor/audience, tool/version, arguments, effects, policy roots, and allowed use. Hash equality checks identity; authentication must come from the trusted issuer/transport mechanism. A signed token, capability use record, and proof receipt are related evidence objects, not aliases for one another.

D Implementation trace and source-to-check map

The private companion contains exact repository revisions and paths. This table retains inspectable symbol names for author review; the final blind artifact should use consistent neutral aliases if these names reveal authorship.

Table D1: Implementation-oriented trace. The table specifies what to record; it is not a claim that an end-to-end run has been performed.

Method	Representative implementation	Inputs to inspect in a reproducible run
Legal routing	CanonicalLegalCorpus registry; rich legal bridge; typed canonical compiler	Dataset revision, source body, jurisdiction, source maps, unsupported facets
Skill import	SkillCenterSkillRecord and read-only bundle adapter	Physical source row, metadata, skill/library Markdown, entry/body/container identities
Intent normalization	IntentIRDocument and formalizer/normalizer ports	Actions, modalities, statements, grounded/inferred status, control edges
CVE adaptation	adapt_cvefixes_candidate; Secu- rityIRFormalizationAdapter	Vulnerable/fixed source records, candidate scope, review binding, prohibition/control polarity
Intent/code correlation	cve_security_gate	Exact intent mappings and independently extracted code effects under one security root
Constraint join	IRConstraintCompiler	Candidate graph, legal/security/intent checks, proof requirements, current roots
Proof authorization	IntentAuthorizationService; AuthorizationQueryComposer; AuthorizationPortfolio	Mandatory job population, backend routes, evidence/verifier identities, terminal decision
Guarded execution	Pre-invocation enforcement wrapper; ExecutionPermit	Enforce mode, current live context, authenticated receipt, expiry/use state, delegate effects
State control	QuackStateClient; StateTransaction	Named command, generation, fence, revision, idempotency and committed result
Protected MCP	AuthorizationGate and shared tool dispatch	Actual handler/tool arguments, envelope, strict UCAN verifier, policy provider, audit/result objects

1. Resolve source and declaration manifests by exact revision/CID.
2. Import skill procedure as data; derive or validate IntentIR.
3. Select applicable LegalIR and reviewed SecurityIR constraints.
4. Bind actual requested tool, arguments, actor and expected effects.
5. Correlate intent-side effects with code/handler-side observations.
6. Compose and execute the required formal jobs; retain non-successes.
7. Produce an exact-context decision, not an effect.
8. Check signed delegation and host-controlled current policy/state.
9. In enforce mode, revalidate and consume the exact permitted use.
10. Dispatch once through the selected protected path.
11. Record observed outcome separately from proof and authorization.
12. Commit operational progression through the DuckDB owner.

508 Failure attribution should name a stage. A bad source extraction, a wrong legal interpretation, an
509 irrelevant CVE analogy, an incomplete effect mapping, a false proof receipt, a stale root, a widened
510 delegation, and a bypassing handler are different defects. The evidence should make it possible to
511 locate which boundary failed rather than attributing all failures to “LLM hallucination.”

512 **E Evaluation completion template**

513 **E.1 Populations and source freezing**

514 Before claiming a measured result, freeze a bounded legal slice, a CVE vulnerable/fixed cohort, a set
515 of SkillCenter records, exact package revisions, native logic fragments, prover/toolchain versions,

model/tokenizer revisions, and sandbox handler versions. Retain license/review state and source failures. Development, calibration, and held-out tasks must be separated by source lineage so that a derivative record of the same source does not leak between splits.

The primary outcome is actual forbidden effects under guarded execution, jointly reported with useful permitted work. A reject-all gate is not useful. Compare the same action candidates and runtime environment under prompt-only, retrieval-assisted, lightweight policy/UCAN, and full proof-oriented enforcement. Then add a closed-loop model agent comparison to measure whether the constraints alter planning and recovery. These answer different questions and should not be pooled.

Table E1: Result placeholders. “Not run” is not zero failures and is not a claim that the repository has no tests.

Measurement	Planned record	Present status
Legal source-to-IR fidelity	Expert-reviewed atoms, exceptions, applicability and source spans	Not run
CVE restriction specificity	Vulnerable positives, fixed negatives, unrelated-code controls	Not run
Skill grounding	Source spans, control-flow fidelity, malicious Markdown negatives	Not run
Proof job execution	Per-family provider/checker route, assumptions, result and receipt	Not run
Guarded effect prevention	Delegate/effect counters under allow, deny, unknown, stale and replay	Existing assertions inspected; frozen run pending
DuckDB owner safety	Wrong generation/fence, concurrent commands, lost reply, restart	Existing implementation and test design inspected; run pending
Cross-transport parity	Stdio/HTTP/P2P same request and outcome; actual network mode	Existing suite inspected; run pending
End-to-end efficiency	All model tokens, context retrieval, proof and state/storage overhead	Not run

E.2 Required counterexamples and ablations

Include omitted legal exceptions, wrong dates/jurisdictions, no applicable legal record, misleading CVE similarity, a fixed negative control, skill text claiming authorization, generated-code effects not declared in the intent, forged proof identifiers, wrong audience, widened path/tenant bounds, expired/revoked tokens, replay, and a changed environment between decision and use. Test each reachable protected route, not just one helper function.

Ablate source provenance, hard applicability, intent/code correlation, proof jobs, current-root binding, capability verification, and consumption separately in a sandbox. Report where the actual effect counter changes. Measure cold and reused evidence, including retrieval/compilation, solver work, clock/root checks, database transactions, storage, and retry overhead. Do not translate graph size or token reduction directly into cost savings.

F Scope, ongoing work, and disclosure

Earlier worktree directives extend this stack with semantic-addressed program worlds, exact/abstract state transitions, learned next-call/next-event/inverse-trace residuals, semantic-preserving refactoring, parallel sealing, and proof-carrying test reuse. They support better candidate generation and incremental checking. Their uncommitted implementation bytes were not available for this revision, so they are not presented as newly executed methods or measured results.

Semantic minification can send a task-specific ContextPack and local aliases rather than entire laws, skill libraries, or repositories. Its source maps, required obligations, and current-root bindings must remain intact. Learned encoders and similarity search nominate candidates; they cannot certify

544 omitted constraints. Separate lexical/token savings, compilation cost, formal-checking cost, and final
545 task utility when this optimization is evaluated.

546 This draft was prepared using language-model assistance for source inspection, organization, and
547 writing. No repository test suite, live model experiment, full corpus recount, or robot execution was
548 performed during this document revision. The authors must replace this disclosure with the exact
549 tools, model versions, human review, and executed experiments used for the final submission.